Department of Computer Science & Engineering
NIT Karnataka, Surathkal

Group – 13

Name – Sarbasish Chowdhury (252IS033)
Name – Rohit Kumar Sah (252IS030)
Name – Md Timsal Alam (252IS021)

M.Tech (CSE-IS)

Subject – Blockchain Architecture Design (CS740)

Title: Design and Implementation of a Time-Window Voting System using Ethereum Smart Contract

Abstract

This project presents the design and implementation of a Time-Window Voting System using Ethereum blockchain technology. The system ensures that voting is restricted to a predefined time interval and only authorized members are allowed to participate. The smart contract enforces strict rules to prevent unauthorized access, double voting, and voting outside the allowed time window. The implementation demonstrates how blockchain can be used to build secure, transparent, and tamper-proof voting systems.

1 Introduction

Traditional voting systems suffer from several limitations such as lack of transparency, centralized control, and vulnerability to manipulation. Blockchain technology addresses these challenges by providing decentralization, immutability, and transparency.

Ethereum, a decentralized platform, enables the execution of smart contracts that automatically enforce rules without human intervention. In this project, a Time-Window Voting System is implemented to demonstrate the application of blockchain in secure voting mechanisms.

2 Objective

The main objectives of this project are:

- To design a decentralized voting system using Ethereum
- To enforce time-based restrictions on voting
- To ensure only registered members can vote
- To prevent duplicate voting
- To compute the final result based on vote counts

3 Technology Stack

- **Blockchain Platform:** Ethereum
- **Smart Contract Language:** Solidity
- **Development Environment:** Remix IDE
- **Version Control:** GitHub

4 System Architecture

The system consists of:

- Smart Contract deployed on Ethereum
- Owner (admin) who controls registration and proposal creation
- Members who participate in voting

5 Problem Statement

The system must satisfy the following conditions:

- Each proposal must have a start time and end time
- Only registered members can vote
- Voting is allowed only if:

$$startTime \leq block.timestamp \leq endTime$$

- Votes outside this time window must be rejected
- Final result should be computed after voting ends

6 Smart Contract Design

6.1 Data Structures

The core structure used in the contract is:

```
struct Proposal {
    string description;
    uint startTime;
    uint endTime;
    uint yesVotes;
    uint noVotes;
    bool exists;
}
```

6.2 State Variables

- `owner`: Address of contract deployer
- `members`: Mapping of registered members
- `proposals`: Mapping of proposal ID to proposal data
- `hasVoted`: Prevents double voting

6.3 Key Functions

6.3.1 registerMember()

Registers a user as a valid voter.

6.3.2 createProposal()

Creates a new proposal with time constraints.

6.3.3 vote()

Allows a member to vote if:

- Voting has started
- Voting has not ended
- User has not voted before

6.3.4 getResult()

Returns result only after voting ends.

7 Implementation

7.1 Contract Deployment

The smart contract was successfully compiled and deployed using Remix IDE.

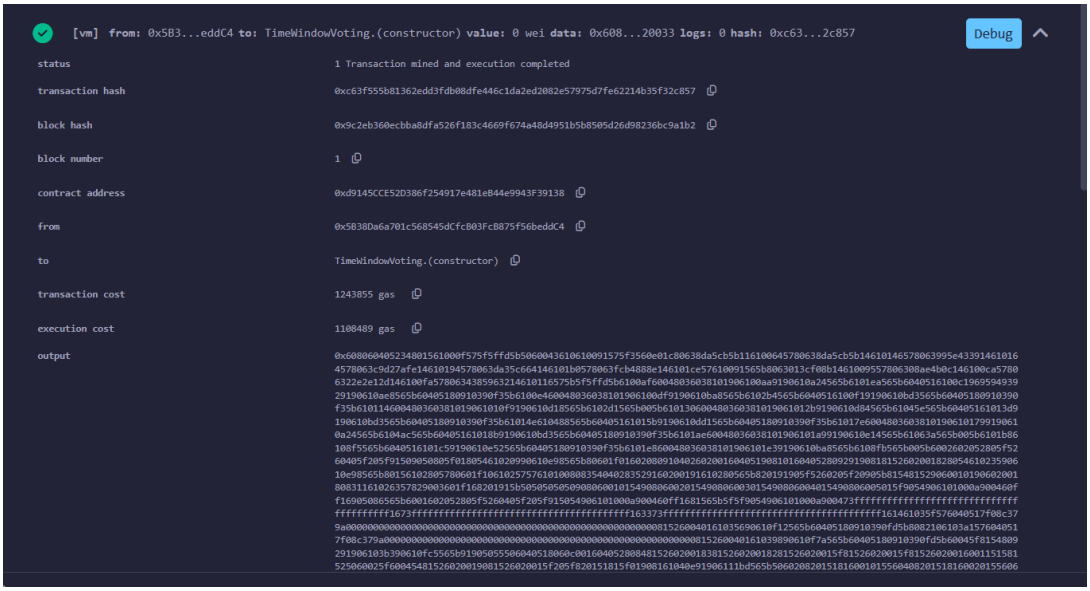

Figure 1: Smart Contract Deployment

7.2 Member Registration

Multiple members were registered using the `registerMember()` function.

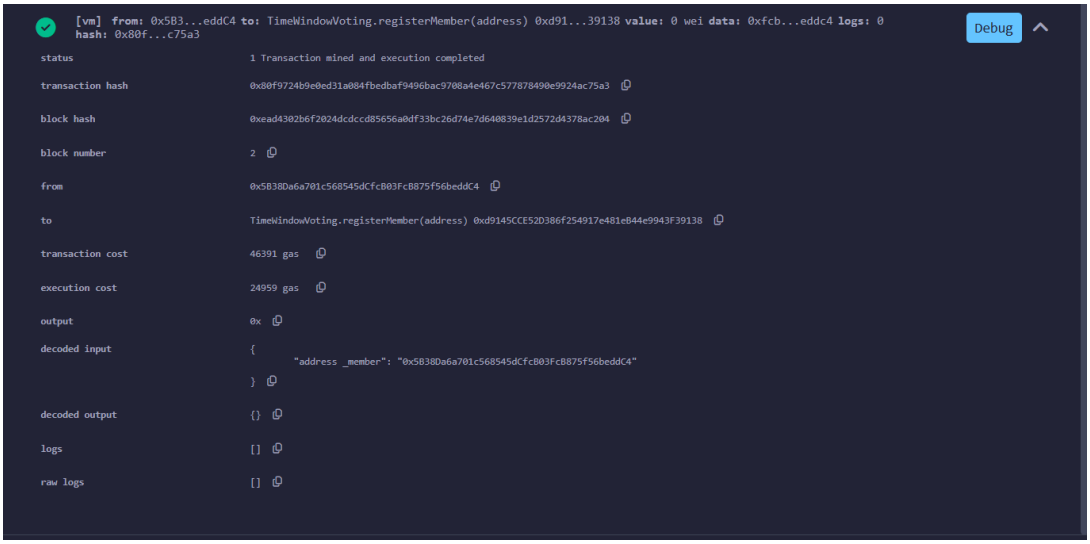

Figure 2: Member Registration 1

Figure 3: Member Registration 2

Figure 4: Member Registration 3

7.3 Proposal Creation

Proposals were created with valid start and end times.

Figure 5: Proposal Creation 1

Figure 6: Proposal Creation 2

Figure 7: Proposal Creation 3

7.4 Voting Process

Votes were successfully cast within the allowed time window.

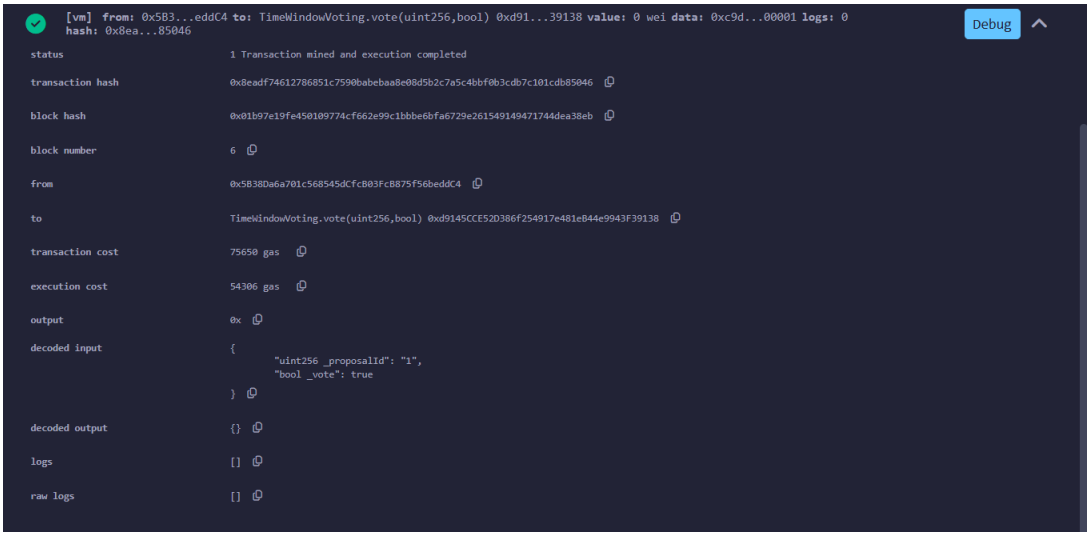

Figure 8: Vote Cast (Yes)

Figure 9: Vote Cast (No)

7.5 Invalid Voting Attempt

An attempt to vote after the voting deadline resulted in failure.

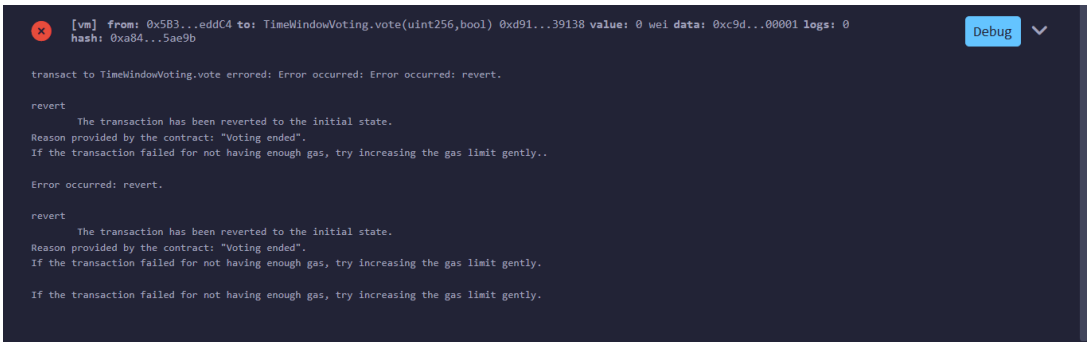

Figure 10: Voting Rejected After Deadline

7.6 Proposal Count Verification

The total number of proposals was verified.

Figure 11: Proposal Count

7.7 Final Result

The result was successfully retrieved after voting ended.

Figure 12: Final Result

8 Results and Analysis

- The system successfully enforced time-based voting restrictions
- Unauthorized voting attempts were rejected
- Double voting was prevented using mapping
- Gas costs were observed for each transaction
- The final result accurately reflected the majority vote

9 Advantages

- Decentralized and tamper-proof system
- Transparent voting mechanism
- Automated rule enforcement

10 Limitations

- Dependent on system time (block timestamp)
- No user-friendly interface
- Requires blockchain knowledge to operate

11 Future Enhancements

- Develop a frontend using Web3.js and React
- Deploy on public testnet (Sepolia)
- Add event logging for better traceability
- Implement role-based access control

12 Conclusion

The project successfully demonstrates how Ethereum smart contracts can be used to build a secure and reliable voting system. The enforcement of time constraints ensures fairness and prevents misuse, making the system suitable for real-world decentralized applications.